Ders İsmi: Veri Yapıları

Ödev No: 1

Hazırlayan;

Adı Soyadı: Ahmet Faruk İkiz

Öğrenci No: b241210040

Dersi Aldığı Grup: 1.Öğretim B Grubu

Dersi Aldığı Öğretim Görevlisi: Dr.Öğr.Üyesi HÜSEYİN DEMİRCİ

İstenenler:

Ödev C++ ile terminalde çalıştırılmak üzere hazırlanacaktır. Çift Yönlü bir bağlı liste içerisinde Tek yönlü bir bağlı liste tutacaktır. Tek yönlü bağlı liste ise içerisinde Şekil sınıfından nesneler tutacaktır. Şekil sınıfından türeyen yıldız, dikdörtgen, üçgen sınıfları olacak bu şekiller polimorfizm ve inheritance ilkelerine göre belirli metotlara ve alanlara sahip olacaktır. Çift yönlü bağlı liste elemanları arasında gezilebilmeli ve bu listenin düğümlerine girilerek şekiller arasında da gezilebilmeli, bu şekiller hareket ettirilebilmelidir. Bu şekiller ve şekil listeleri silinebilmelidir. Program kapatıldığında, bütün düğümler ve şekillerin nitelikleri bilgisayar üzerine kaydedilmeli, program yeniden açıldığında bunlar geri yüklenebilmelidir.

Ödevde Yaptıklarım ve Kurduğum Mimari:

Programda kullandığım nesne ve fonksiyonları genel olarak CizimProgrami isimli bir sınıf yönetmektedir.

Main.cpp :

İçerisinde CizimProgrami nesnesi heap alanında oluşturulur ve baslat() metodu çağırılır. rand() fonksiyonu, srand(time(NULL)),için ayarlanır.

Ekran.hpp ve Ondan Türeyen Sınıflar:

Ödevimde terminal alanını 3 parça halinde düşündüm (Sol Menü, Alt Menü, Canvas sınıfları) sonrasında bu 3 parçayı içinde tutacak AnaEkran sınıfını oluşturdum. Her bir sınıf içerisinde 2 boyutlu bir char dizisi tutar. Bu sınıfların kendi Ekran dizilerini yönetecek Guncelle() metotları vardır ve son olarak bu 3 sınıf AnaEkran sınıfının Entegre() fonksiyonuna argüman olarak yollanır, burada birleştirilirler. Bu 4 sınıf Ekran sınıfından miras alırlar.

CizimProgrami.hpp:

Programın en temel sınıflarından biridir. Ekran.hpp içerisindeki sınıfları heap'te oluşturur ve yönetir. Menü işlemleri ve şekil çizdirme fonksiyonlarını yönetir. Programın temel taşı olan çift ve tek yönlü liste burada oluşturulur ve yönetilir. Programın çalışması, yönetilmesi ve kapanırken kaydedilmesinden sorumludur. AnaEkraniCiz() metodu "cout << _anaEkran;" yapar, özel tanımlanmış << operatörü ise AnaEkranın dizisini bir strine aktarır ve erkana basar. DosyayaYaz() ve DosyadanOku() metotlarıyla kayıt sistemini yönetir.

Sekil.hpp ve Ondan Türeyen Sınıflar:

Her şekil kendini çizdirmek/tasarlamak için 20x20 boyutunda bir diziye ve Cizdir() fonksiyonuna sahiptir. Bunlar daha sonra Canvas sınıfının Guncelle() metoduna gönderilen ŞekilListesi üzerinden x, y kordinatlarına göre canvasa yerleştirilirler. Şekillerin hareket etmesi kendi içerisindeki sag(), sol() gibi metotlarla sağlanmıştır.

Listeler:

*Listelerde dolaşırken gömülü bir iteratör sınıfı yazdım. Tek yönlü listede geri gezinmek için iteratörü baştan başlatıp bir önceki düğüme kadar yürüttüm.

Öğrendiklerim:

*Programımda birkaç kez "Circular Dependency" hatasıyla karşılaşmam sonucu "Forward Declaration" çözümünü keşfettim.

*Öncesinde ana ekranı bastırırken her karakter için teker teker cout çağırıyordum. Bu da rami çok yoruyordu. Ana ekran dizisini bir stringe taşıyıp cout << string; yapma fikrini denerken bunu daha da verimli hale getiren reserve() fonksiyonunu keşfettim. Bunu gerçekleştirirken AnaEkran sınıfı için << operatörünü override ettim.

*Iteratör mantığını daha iyi kavradım. Programımda gömülü bir iteratör kullanmam sayesinde iteratör yazımı ve kullanımı konusunda pratik yapmış oldum.

*Mingw ve makefile ile derleme ve çalıştırma pratiği edindim.

*Z sıralamasını yaparken bubble sort algoritması kullandım.

Zorlandığım Kısımlar:

*Şekillerin çizilmesi ve canvas'a aktarılması en çok zorlandığım kısım oldu. İlk olarak her şekil listesinin kendisine ait canvas'ı olduğu bir mimari tasarladım fakat bu mimariyi uygulaması oldukça zordu çünkü Şekil Listesinin canvası vardı fakat canvasın da kendi içinde o listenin şekillerine ihtiyacı vardı. Ben de sol menü ve alt menü de yaptığım mimariyi uygulamaya karar verdim. CizimProgrami içinde tek bir canvas oluşturdum, gerektiğinde bu canvas'a şekil listesini gönderip şekilleri çizdirdim.

*Pointer ve heap hafızası yönetimi bir diğer zorlandığım şey oldu. Bazı noktalarda program alakasız yerlerde hata verdi, debug süreci saatler aldı.

*Eğer ödevi kendim tasarlamış olsaydım Şekil Listesini de çift yönlü yapardım. Bu sayede şekiller arası ileri geri dolaşmak daha kolay olurdu. Ayrıca iki listeyi de circular yapmak kullanıcı deneyimini oldukça iyileştirecektir.

*Şekil silme işlemleri ve çıkış tuşunu ayalarken iteratör ve liste yönetimi konusunda pek çok sorun yaşadım. Bazı nesnelerin destructorlarını tekrar ayarlamam gerekti.